

Synthesis and Augmentation for Structured data

Minh Le et al.
Advisor: Dr. Thuy Do
Department of Computer Science
Luther College

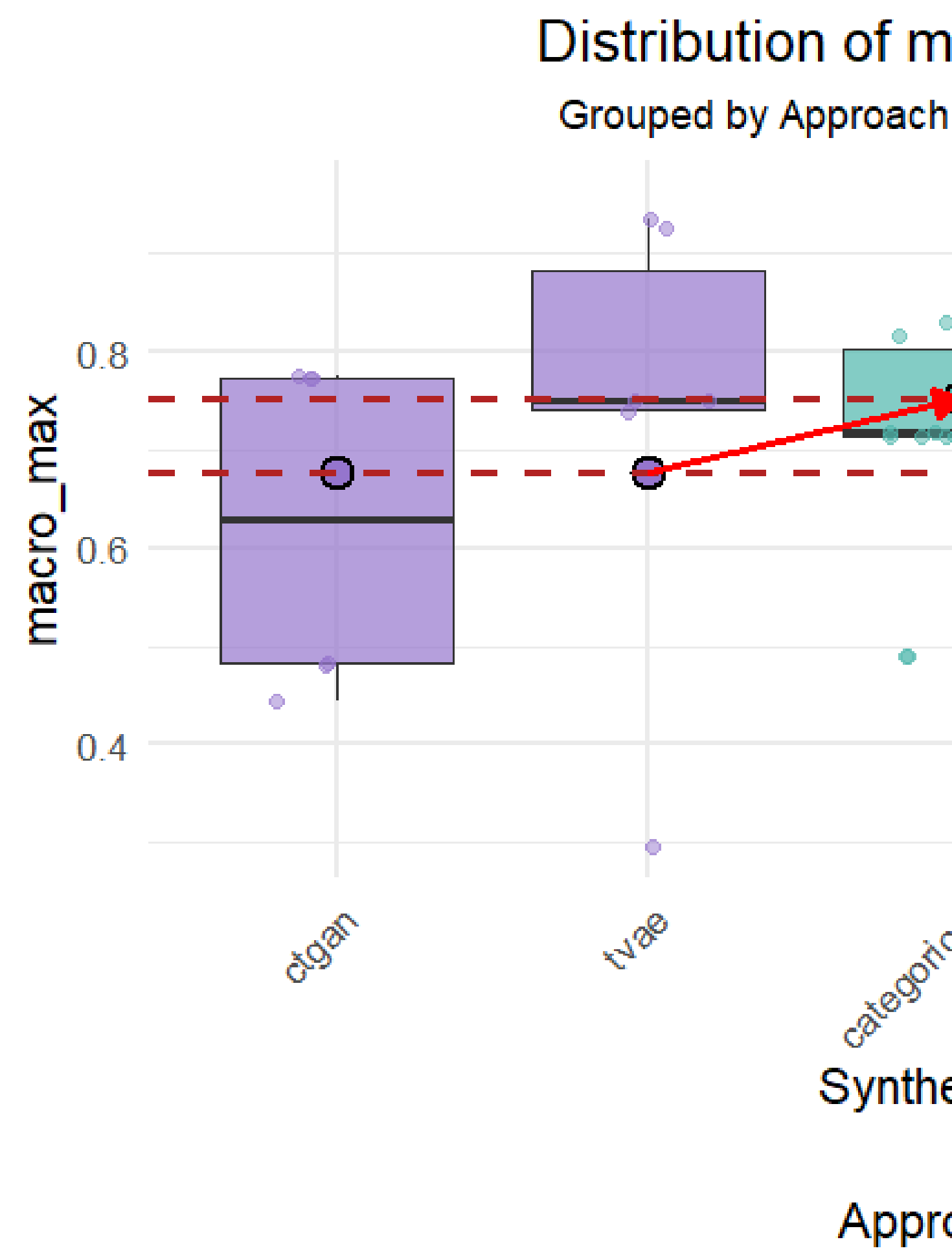
Result

We fine-tuned an object detection model specialized in detecting sheep using YOLOv8.

Using the number of sheep in the pen as an invariant, we developed an algorithm using vector similarity search to map the sheep from one frame to the next.

We transformed 2D bounding box coordinates from detected images to 3D physical coordinates using vector decomposition and infinitesimal approximation.

Results



On average, the new approach is better than the old approach, and the new synthesizers, regardless of the data, outperform training the X_synthesizer.

Across 6 datasets, 6, the new X_synthesizer (ctgan or tvae) increases the result only marginally, and the effect is not statistically significant.

→ Don't include y in X_input and time

→ Generate X' and y' separately

Conclusion and Future

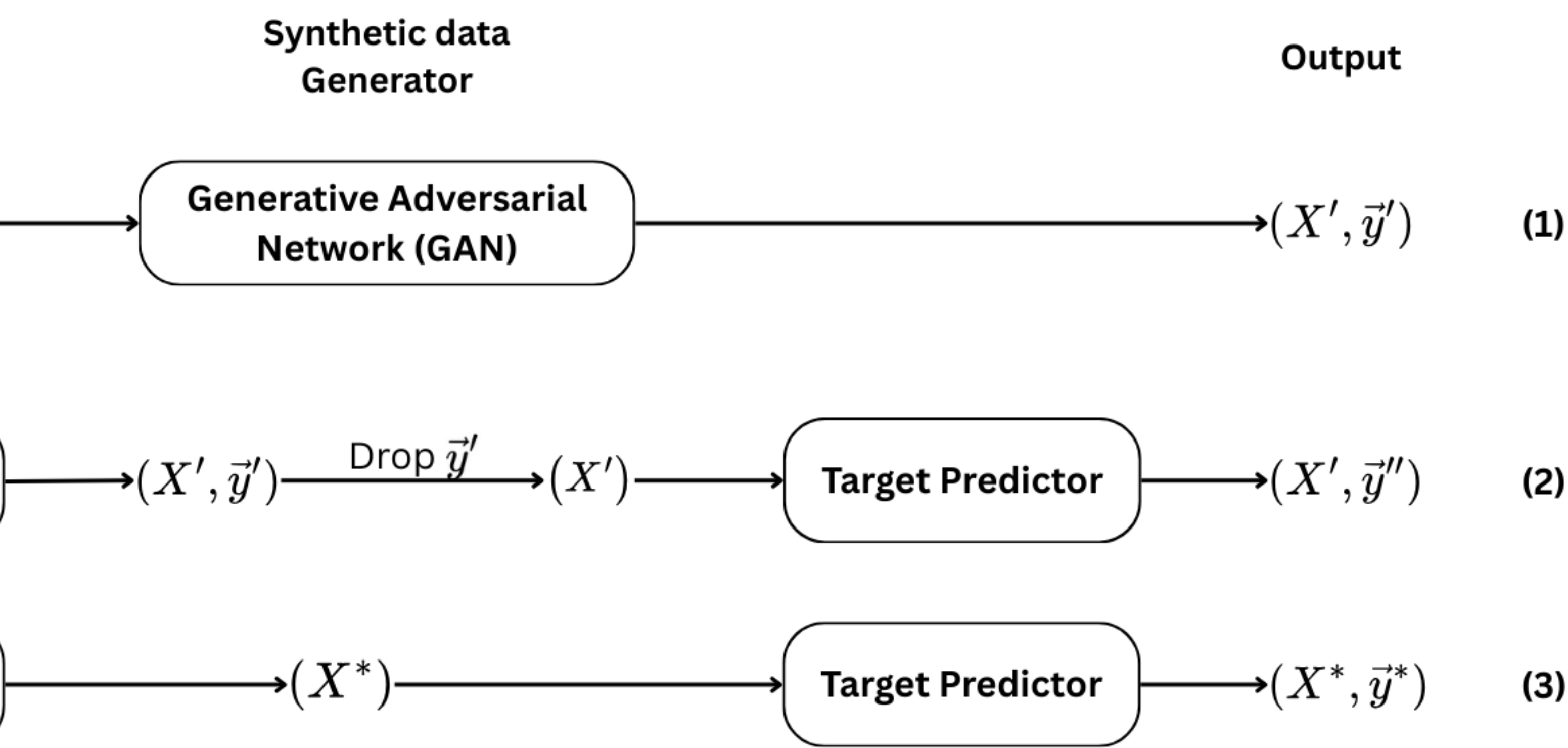
Run more datasets

Expand for regression

improve X_synthesizer (ctgan or tvae)

References

1. Xu, Lei, et al. "Modeling tabular data with generative adversarial networks." *Information processing systems* 32 (2018): 2857-2866.
2. Chen, Tianqi, and Carlos Guestrin. "XGBoost: A scalable tree boosting system." *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016.
3. Breiman, Leo. "Random forests." *Machine learning* 45 (2001): 5-32.



Synthesis and Augmentation for Structured Data

Minh Le et al.
Advisor: Dr. Thuy Do
Department of Computer Science
Luther College

Introduction

Generating high-fidelity synthetic tabular data is challenging due to mixed data types (discrete/continuous) and complex column distributions (multimodal, imbalanced). While generative models like GANs (e.g., CTGAN, TVAE) have advanced the field, a gap often remains between synthetic and real data quality. This work aims to improve the realism of generated structured data.

Data & Evaluation

We evaluate synthetic data quality using its practical utility in a ML task across 6 datasets.

1. A Deep Neural Network (DNN) is trained only on the generated synthetic data.
2. The DNN is then tested on unseen real data (the test set).
3. We measure this performance using the F1-macro score, where higher scores (max 1) indicate better accuracy.

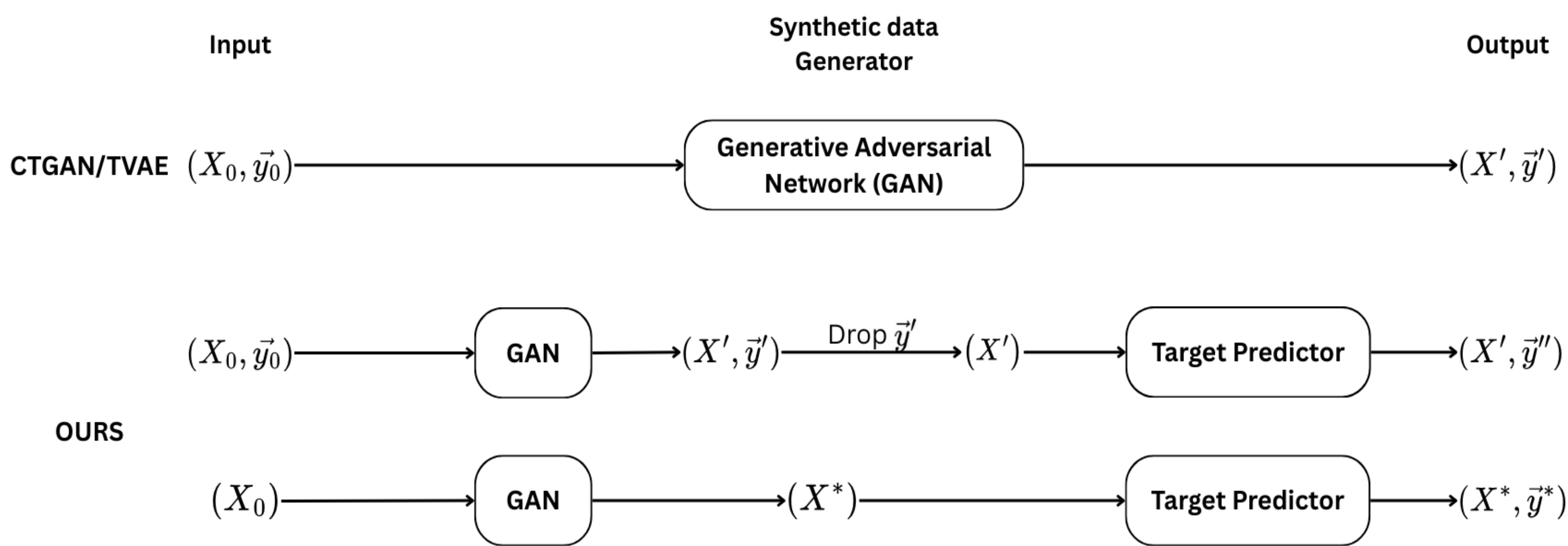
Strong performance by the DNN on real data signifies that the synthetic data realistically captures the original data's patterns.

Method

We propose a decoupled/composite synthesis pipeline, separating feature (X_{syn}) and target (y_{syn}) generation, to replace the current modulus pipeline outlined in Xu et al. [4].

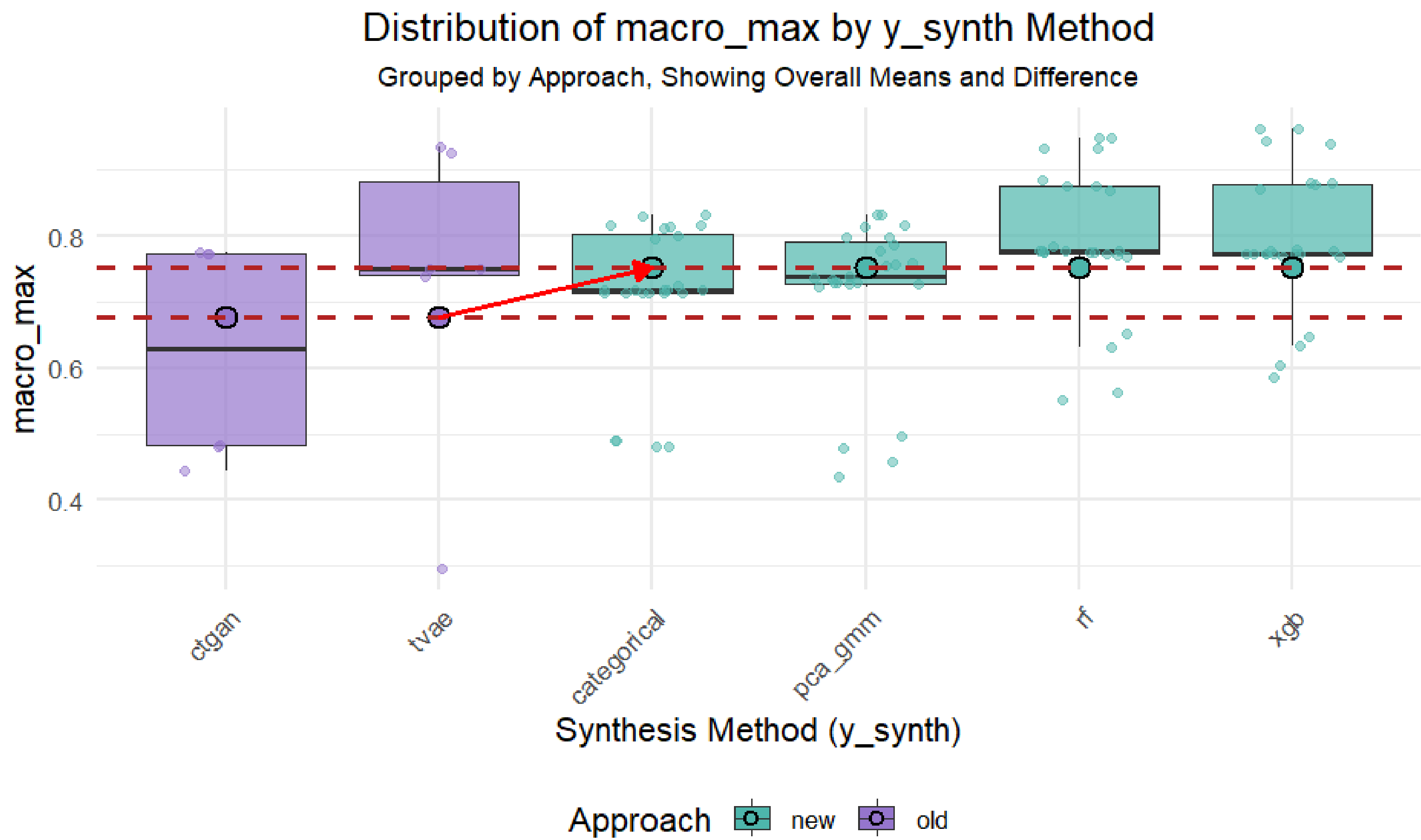
Additionally, we investigate the impact of excluding the target y during the training of the feature generator ($X_{\text{synthesizer}}$) on downstream performance and resource usage.

1. Generate X_{syn} using a chosen generative model (e.g., CTGAN, TVAE) trained on real features X_0
2. Train a predictive model (e.g., XGBoost, Random Forest) on the original (X_0, y_0) data
3. Use the trained predictor to generate y_{syn} based on X_{syn}



Data Synthesis Pipelines Flowchart

Results



On average, the new approach (ours) is statistically better ($p_value < .0001$) than the old approach by 7.36% across 6 datasets, 2 X synthesizers, regardless of whether y was included in training the $X_{\text{synthesizer}}$.

Across 6 datasets, 6 y synthesizers, regardless of $X_{\text{synthesizer}}$ (ctgan or tvae), including y in the X_{input} increases the result only marginally (0.3%). However, this effect is not statistically significant ($p_value = 0.5091 > 0.05$).

Conclusion

- Generating X_{syn} & y_{syn} separately will result in synthetic data that more closely resemble distribution and properties of the real data, demonstrated through the higher F1-macro scores
- Don't include y in X_{input} to save computational power and time

Future Work

- Run more datasets
- Expand for regression
- improve $X_{\text{synthesizer}}$ (using Fourier features, etc.)

References

1. Breiman, Leo. "Random forests." *Machine learning* 45 (2001): 5-32.
2. Chen, Tianqi, and Carlos Guestrin. "Xgboost: A scalable tree boosting system." *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016.
3. Goodfellow, Ian, et al. "Generative adversarial networks." *Communications of the ACM* 63.11 (2020): 139-144.
4. Xu, Lei, et al. "Modeling tabular data using conditional gan." *Advances in neural information processing systems* 32 (2019).

Synthesis and Augmentation for Structured Data

Minh Le et al.
Advisor: Dr. Thuy Do
Department of Computer Science
Luther College

Introduction

Generating high-fidelity synthetic tabular data is challenging due to complex column distributions (multimodal, imbalanced) and mixed data types (discrete/continuous). While generative models like GANs (e.g., CTGAN, TVAE) have advanced the field, a gap often remains between synthetic and real data quality. This work aims to improve the realism of generated synthetic structured data.

Data & Evaluation

We evaluate synthetic data quality using its practical utility in a ML task across 6 datasets.

1. A Deep Neural Network (DNN) is trained only on the generated synthetic data.
2. The DNN is then tested on unseen real data (the test set).
3. We measure this performance using the F1-macro score (ranging from 0 to 1), where higher scores indicate better recall & precision.

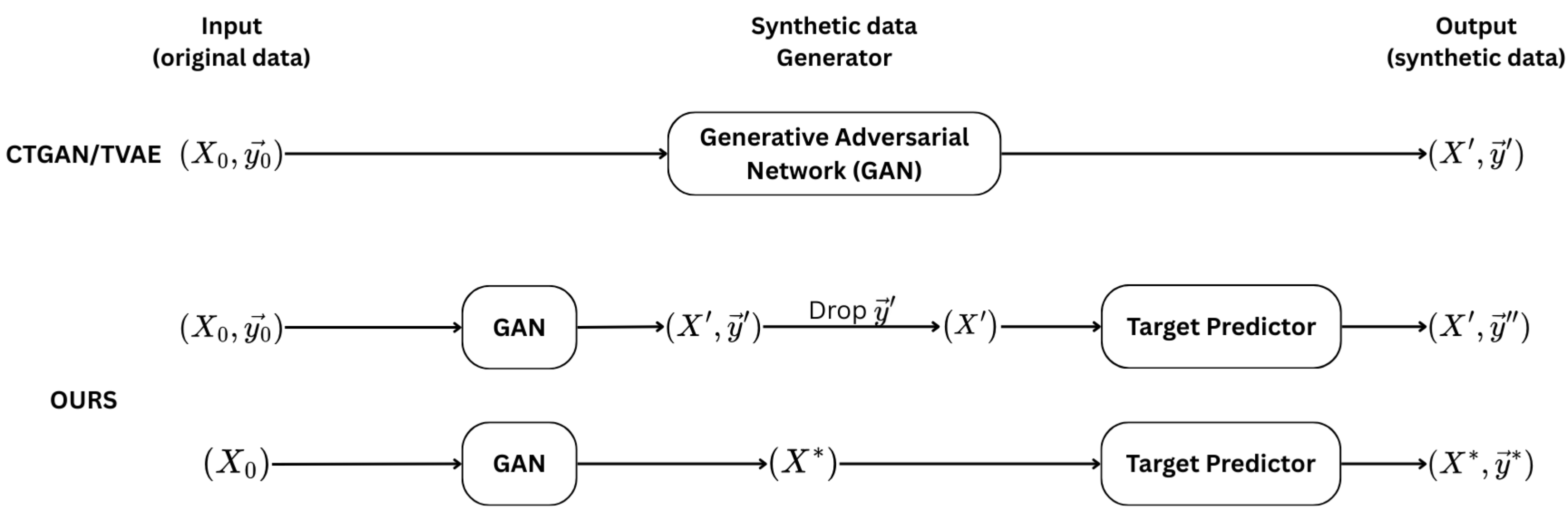
Strong performance by the DNN on real data signifies that the synthetic data realistically captures the original data's patterns.

Method

We propose a **decoupled synthesis pipeline**, separating feature (X_{syn}) and target (y_{syn}) generation, to replace the current modulus pipeline outlined in Xu et al. [4].

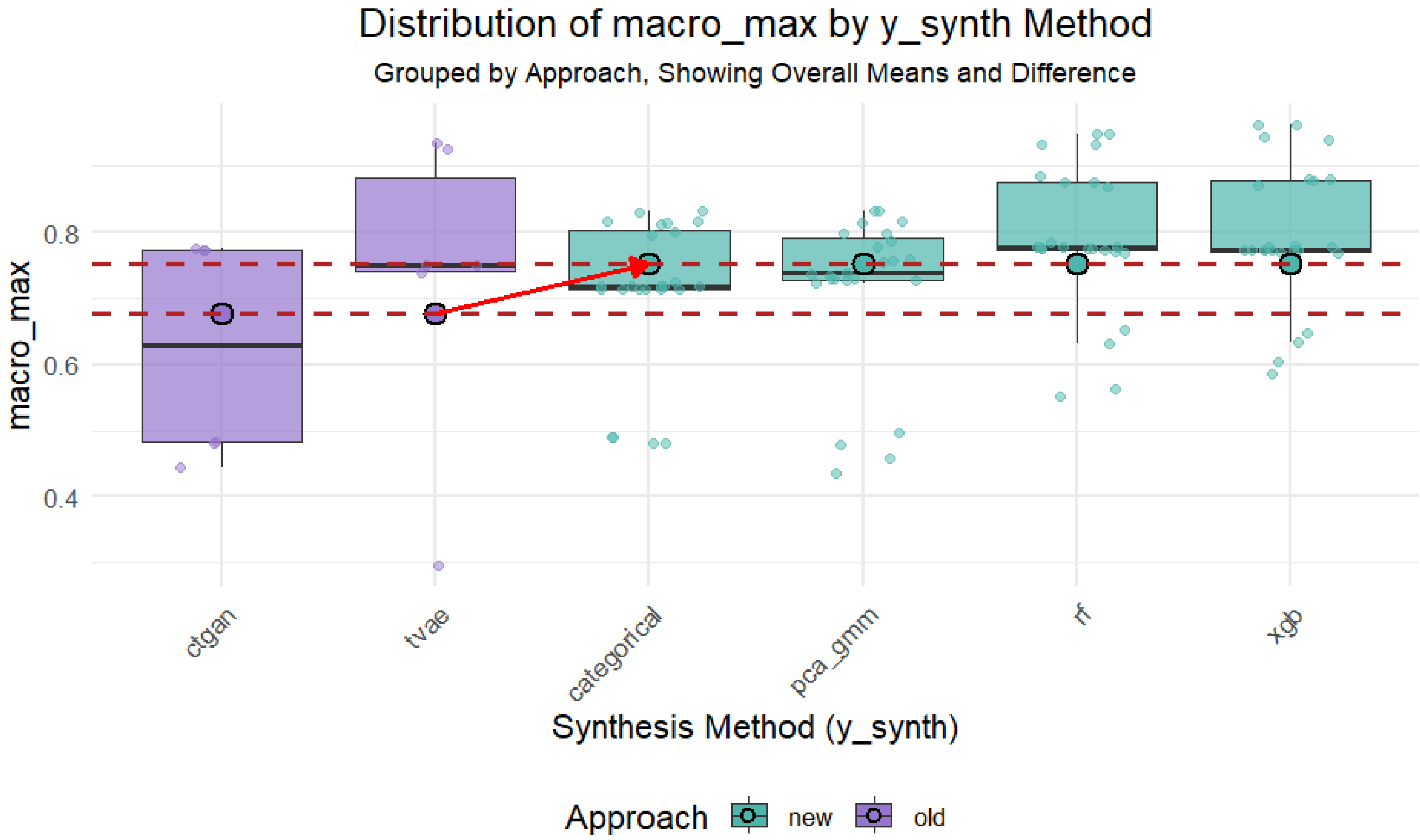
Additionally, we investigate the impact of excluding the target y during the training of the feature generator ($X_{synthesizer}$) on downstream performance and resource usage.

1. Generate X_{syn} using a chosen generative model (e.g., CTGAN, TVAE) trained on real features X_0
2. Train a predictive model (e.g., XGBoost, Random Forest) on the original (X_0, y_0) data
3. Use the trained target predictor to generate y_{syn} based on X_{syn}



Data Synthesis Pipelines Flowchart

Results



Our decoupled synthesis approach significantly improves upon standard end-to-end generation:

- Achieved an average F1-macro improvement of 0.0736 (**10.86% relative increase**) over baseline. This significant gain was consistent across 6 datasets and various generative models
- Including target y during feature X synthesis offered negligible benefit, yielding only a marginal (+0.3% F1-macro) and **statistically insignificant** ($p=0.5091$) improvement. This indicates **potential for model simplification** without performance loss

Conclusion

- **Decoupled Synthesis Enhances Utility:** Generating features (X_{syn}) and targets (y_{syn}) separately produces higher-quality synthetic data compared to end-to-end methods, evidenced by significantly improved downstream F1-macro scores.
- **Optimize Feature Generation:** Excluding the target variable (y) during feature synthesis is recommended; it offers computational savings with negligible impact on final model performance.

Future Work

More Validation: Evaluate the proposed pipeline across a more extensive and diverse range of datasets.

Task Expansion: Adapt and test the decoupled approach for regression tasks, not just classification.

$X_{synthesizer}$ Research: Implement advanced techniques (e.g., incorporating Fourier features) to further improve the quality of the feature synthesizer.

References

1. Breiman, Leo. "Random forests." *Machine learning* 45 (2001): 5-32.
2. Chen, Tianqi, and Carlos Guestrin. "Xgboost: A scalable tree boosting system." *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016.
3. Goodfellow, Ian, et al. "Generative adversarial networks." *Communications of the ACM* 63.11 (2020): 139-144.
4. Xu, Lei, et al. "Modeling tabular data using conditional gan." *Advances in neural information processing systems* 32 (2019).